

Объектно-ориентированное программирование

Никита Остроухов

Кафедра информатики МФТИ

Физика столкновений

Добавление второго шара

```
int main()
{
    int x = 0, y = 0, vx = 0, vy = 0, r = 0;

    ...

    for (;;)
    {
        moveBall(&x, &y, &vx, &vy, dt);

        controlBall(&vx, &vy);

        drawBall(x, y, r, R, G, B);
    }

    ...
}
```

Пора в нашу программу **добавить** еще
один **шар**

Добавление второго шара

```
int main()
{
    int x1 = 0, y1 = 0, vx1 = 0, vy1 = 0, r1 = 0;
    int x2 = 0, y2 = 0, vx2 = 0, vy2 = 0, r2 = 0;
    ...

    for (;;)
    {
        moveBall(&x1, &y1, &vx1, &vy1, dt);

        controlBall(&vx1, &vy1);

        drawBall(x1, y1, r1, R1, G1, B1);
    }

    ...
}
```

Добавим второй **набор переменных** для
другого шара



Добавление второго шара

```
int main()
{
    int x1 = 0, y1 = 0, vx1 = 0, vy1 = 0, r1 = 0;
    int x2 = 0, y2 = 0, vx2 = 0, vy2 = 0, r2 = 0;
    ...

    for (;;)
    {
        moveBall(&x1, &y1, &vx1, &vy1, dt);
        moveBall(&x2, &y2, &vx2, &vy2, dt);

        controlBall(&vx1, &vy1);

        drawBall(x1, y1, r1, R1, G1, B1);
        drawBall(x2, y2, r2, R2, G2, B2);
    }
    ...
}
```

И сделаем шар **неуправляемым** для игрока,
не добавляя вызов функции **controlBall**

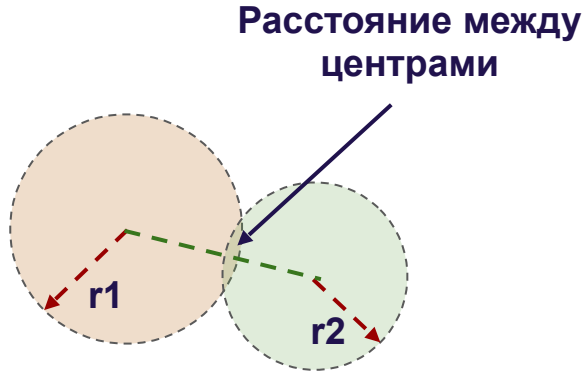
Чтобы добавить еще один шар
мы добавили **всего 3 строки**

Чтобы добавить еще один шар
мы добавили **всего 3 строки**

Функции - это круто

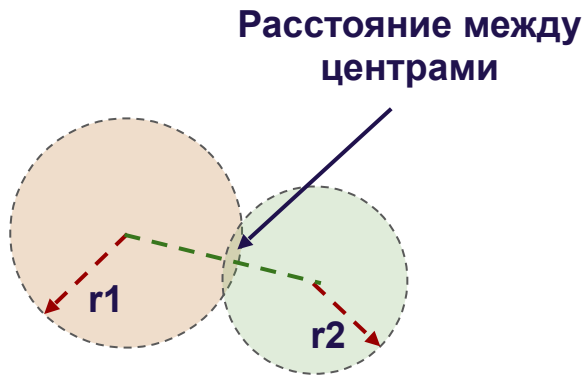
Столкновение шаров

Условие столкновения



Если **расстояние** между центрами шаров
меньше суммы радиусов - шары
столкнулись

Условие столкновения



Если **расстояние** между центрами шаров
меньше суммы радиусов - шары
столкнулись

$\text{distance} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

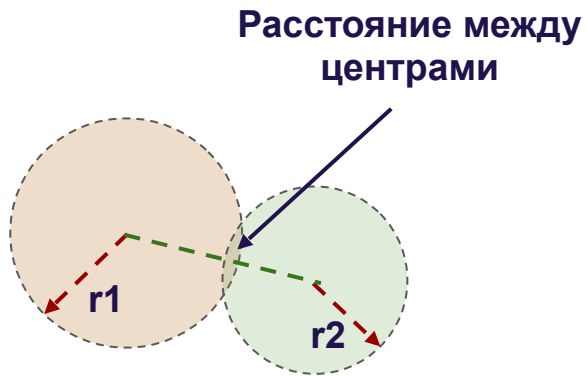
if (**distance** < $r_1 + r_2$)

has collision

else

no collision

Условие столкновения



$\text{distance} = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$

```
if (distance < r1 + r2)
    has collision
else
    no collision
```

Напишем функцию `hasCollision`, которая возвращает результат проверки

Условие столкновения

```
bool hasCollision(int x1, int y1, int r1,  
                  int x2, int y2, int r2)  
{  
  
    ...  
  
}
```

Напишем функцию **hasCollision**, которая **возвращает** результат проверки

Условие столкновения



```
bool hasCollision(int x1, int y1, int r1,  
                 int x2, int y2, int r2)  
{  
  
    ...  
  
}
```

Тип возвращаемого значения **bool** - потому
что функция **будет возвращать** true или false

Условие столкновения

```
bool hasCollision(int x1, int y1, int r1, Из cmath.h  
                  int x2, int y2, int r2)  
{  
    float distance = sqrt((x1 - x2) * (x1 - x2) +  
                           (y1 - y2) * (y1 - y2));  
  
    if (distance < r1 + r2)  
        return true;  
  
    return false;  
}
```

Реализация следует из рисунка

Условие столкновения

```
bool hasCollision(int x1, int y1, int r1,  
                 int x2, int y2, int r2)  
{  
    float distance = sqrt((x1 - x2) * (x1 - x2) +  
                          (y1 - y2) * (y1 - y2));  
  
    return distance < r1 + r2;  
}
```

Более простая запись кода

Столкновение шаров

```
...  
  
for (;;)   
{  
    moveBall(&x1, &y1, &vx1, &vy1, dt);  
    moveBall(&x2, &y2, &vx2, &vy2, dt);  
  
    controlBall(&vx1, &vy1);  
  
  
    drawBall(x1, y1, r1, R1, G1, B1);  
    drawBall(x2, y2, r2, R2, G2, B2);  
}  
...
```

Вернемся к прошлому коду без столкновений

Столкновение шаров

```
...  
  
for (;;)   
{  
    moveBall(&x1, &y1, &vx1, &vy1, dt);  
    moveBall(&x2, &y2, &vx2, &vy2, dt);  
  
    controlBall(&vx1, &vy1);  
  
    if (hasCollision(x1, y1, r1, x2, y2, r2))  
        resolveCollision(...)  
  
    drawBall(x1, y1, r1, R1, G1, B1);  
    drawBall(x2, y2, r2, R2, G2, B2);  
}  
...
```

Добавим столкновения шаров

Столкновение шаров

```
...  
  
for (;;)   
{  
    moveBall(&x1, &y1, &vx1, &vy1, dt);  
    moveBall(&x2, &y2, &vx2, &vy2, dt);  
  
    controlBall(&vx1, &vy1);  
  
    if (hasCollision(x1, y1, r1, x2, y2, r2))  
        resolveCollision(...)   
  
    drawBall(x1, y1, r1, R1, G1, B1);  
    drawBall(x2, y2, r2, R2, G2, B2);  
}  
...
```

Функция `resolveCollision` пересчитывает скорости шаров по закону сохранения импульса

Задание

Реализовать функцию `resolveCollision`

Задание

Сделать одну из следующих игр:

1. **Догони меня, кирпич** - ты должен догнать один шарик, уворачиваясь от другого
2. **Погоня** - пройти как можно более длинный путь, пока в тебя не врежутся
3. **PaintBall** - граф. редактор, где ты рисуешь шариками (а другие шарики это стирают)
4. **Хулиган** - пихать другие шарики, у кого больше скорость, тот победил. Если тебя пихнули с большей скоростью, чем у тебя - ты проиграл.
5. **Чернобыль**: ты - быстрый нейтрон, подлетаешь к ядрам и они взрываются. Или ты - замедлитель: какой-то шарик меняет цвет, к нему надо подбежать и замедлить реакцию, а то он взорвется

Рефакторинг

...

```
int main()
```

```
{
```

```
    int x1 = 0, y1 = 0, vx1 = 0, vy1, r1 = 0,  
        R1 = 0, G1 = 0, B1 = 0;
```

```
    int x2 = 0, y2 = 0, vx2 = 0, vy2, r2 = 0,  
        R2 = 0, G2 = 0, B2 = 0;
```

...

```
drawBall(x1, y1, r1, R1, G1, B1);
```

```
drawBall(x2, y2, r2, R2, G2, B2);
```

...



Наши функции принимают по **6 аргументов**
Работать с ними - **неудобно**

Рефакторинг

...

```
int main()
```

```
{
```

```
    int x1 = 0, y1 = 0, vx1 = 0, vy1, r1 = 0,  
        R1 = 0, G1 = 0, B1 = 0;
```

```
    int x2 = 0, y2 = 0, vx2 = 0, vy2, r2 = 0,  
        R2 = 0, G2 = 0, B2 = 0;
```

...

```
drawBall(x1, y1, r1, R1, G1, B1);
```

```
drawBall(x2, y2, r2, R2, G2, B2);
```

...



Это **переменные** относящиеся к одному **объекту** -
объекту шара

Как от этого избавиться?

Структуры

```
int main()
{
    int x = 0;
    int y = 0;

    x = 2;
}
```

В языке C++ мы можем объединять переменные в структуры

Структуры

```
struct Vector2f  
{  
    int x = 0;  
    int y = 0;  
};
```



```
int main()  
{  
    int x = 0;  
    int y = 0;  
  
    x = 2;  
}
```

Мы описали **структуру** Vector2f которая **объединяет** две **integer** переменные

Структуры

```
struct Vector2f  
{  
    int x = 0;  
    int y = 0;  
};
```



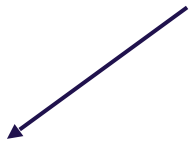
```
int main()  
{  
    int x = 0;  
    int y = 0;  
  
    x = 2;  
}
```

Переменные которые объединены в структуру называются **полями структуры**

Структуры

```
struct Vector2f  
{  
    int x = 0;  
    int y = 0;  
};
```

```
int main()  
{  
    Vector2f vect;  
  
    vect.x = 2;  
}
```



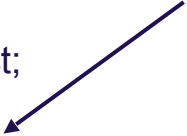
Структура - пользовательский **тип данных**

Структуры объявляются также как и
остальные типы переменных

Структуры

```
struct Vector2f  
{  
    int x = 0;  
    int y = 0;  
};
```

```
int main()  
{  
    Vector2f vect;  
    vect.x = 2;  
}
```



Обратиться к полям структуры можно через оператор точка

Рефакторинг

```
struct Ball  
{  
    ...  
}
```



```
int main()  
{  
    int x1 = 0, y1 = 0, vx1 = 0, vy1 = 0, r1 = 0,  
        R1 = 0, G1 = 0, B1 = 0;  
    int x2 = 0, y2 = 0, vx2 = 0, vy2 = 0, r2 = 0,  
        R2 = 0, G2 = 0, B2 = 0;  
  
    ...  
  
    drawBall(x1, y1, r1, R1, G1, B1);  
    drawBall(x2, y2, r2, R2, G2, B2);  
  
    ...  
}
```

Мы можем **объединить переменные** относящиеся
к шару в **структуру Ball**

Рефакторинг

```
struct Ball
{
    int x = 0;
    int y = 0;
    int vx = 0;
    int vy = 0;
    int r = 0;
    int R = 0;
    int G = 0;
    int B = 0;
};
...
```



**Опишем все переменные структуры
относящиеся к шару**

Рефакторинг

```
struct Ball
{
    int x = 0;
    int y = 0;
    int vx = 0;
    int vy = 0;
    int r = 0;
    int R = 0;
    int G = 0;
    int B = 0;
};
```

...

```
int main()
```

```
{
```

```
    Ball ball1;
```

```
    Ball ball2 = {400, 300, 1, 2, 50, 5, 5, 5};
```

```
    ...
```

Теперь мы можем объявить переменные
с типом Ball

Рефакторинг

```
struct Ball
{
    ...
};

int main()
{
    Ball ball1;
    Ball ball2;
    ...

    drawBall(ball1.x, ball1.y, ball1.r, ball1.R,
              ball1.G, ball1.B);
    drawBall(ball2.x, ball2.y, ball1.r, ball1.R,
              ball1.G, ball1.B);

    ...
}
```



И обращаться к полям внутри структуры

Рефакторинг

```
struct Ball
{
    ...
};

int main()
{
    Ball ball1;
    Ball ball2;
    ...

    drawBall(ball1);
    drawBall(ball2);

    ...
}
```



Мы можем переписать функцию drawBall так чтобы она принимала структуру как аргумент вместо 6 переменных

Функция drawBall

```
void drawBall(int x, int y, int r, int R, int G, int B)
{
    for (int i = 0; i < r; i++)
    {
        txSetFillColor (RGB(R * i / r, G * i / r,
                               B * i / r));
        txSetColor      (RGB(R * i / r, G * i / r,
                               B * i / r));
        txCircle (x + i / 2, y - i / 2, r - i);
    }
}
```

Перепишем функцию с учетом изменения аргументов

Функция drawBall

```
void drawBall(Ball ball)
{
    for (int i = 0; i < ball.r; i++)
    {
        txSetFillColor (RGB(ball.R * i / ball.r,
                             ball.G * i / ball.r, ball.B * i / ball.r));
        txSetColor      (RGB(ball.R * i / ball.r,
                             ball.G * i / ball.r, ball.B * i / ball.r));
        txCircle (ball.x + i / 2, ball.y - i / 2, ball.r - i);
    }
}
```

Перепишем функцию с учетом изменения аргументов

Рефакторинг функции moveBall

Функция drawBall

```
void moveBall(int* x, int* y, int vx, int vy, float dt)
{
    *x = *x + vx * dt;
    *y = *y + vy * dt;
}
```

Функция drawBall

Копирование



```
void moveBall(Ball ball, float dt)
{
    ball.x = ball.x + ball.vx * dt;
    ball.y = ball.y + ball.vy * dt;
}
```

У нас опять возникает проблема копирования

Функция drawBall

Нет копирования



```
void moveBall(Ball* ball, float dt)
{
    *ball.x = *ball.x + *ball.vx * dt;
    *ball.y = *ball.y + *ball.vy * dt;
}
```

Можем передавать адрес где лежит переменная ball, чтобы избежать копирования

Функция drawBall

```
void moveBall(Ball* ball, float dt)
{
    *ball.x = *ball.x + *ball.vx * dt;
    *ball.y = *ball.y + *ball.vy * dt;
}
```

Синтаксическая
ошибка



Оператор точка приоритетнее оператора
звездочка - сначала произойдет обращение к
полям, а только потом обращение по адресу

Функция drawBall

```
void moveBall(Ball* ball, float dt)
{
    (*ball).x = (*ball).x + (*ball).vx * dt;
    (*ball).y = (*ball).y + (*ball).vy * dt;
}
```

Чтобы этого избежать можно изменить приоритет операций

Функция drawBall

```
void moveBall(Ball* ball, float dt)
{
    ball->x = ball->x + ball->vx * dt;
    ball->y = ball->y + ball->vy * dt;
}
```

То же самое

Задание

Завершить рефакторинг оставшейся части кода